

CO5_ASSESSMENT 2

PSEUDOCODE WRITING TASK

1. Real-Time Face Detection and Recognition

A security system must process a live camera stream to:

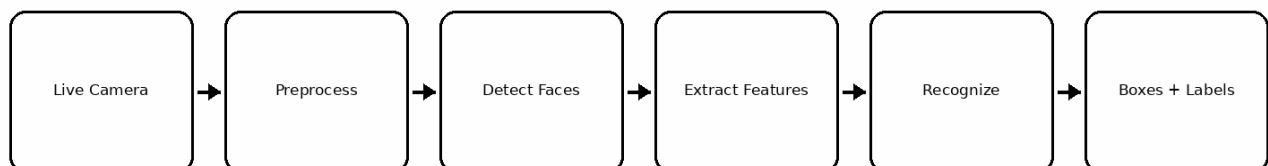
- Capture video frames continuously.
- Detect multiple faces.
- Recognize registered persons using a trained dataset.
- Display bounding boxes and identity labels.

Constraints:

- Must operate in real time.
- Processing delay should be minimized.
- Multiple faces must be handled simultaneously.

Write detailed pseudocode for the complete OpenCV-based face detection and recognition pipeline. Include image acquisition, preprocessing, face detection, feature extraction, recognition, visualization, and efficient loop control.

REAL-TIME FACE DETECTION & RECOGNITION



Illustrative OpenCV pipeline

PSEUDOCODE:

BEGIN

1. INITIALIZATION

- Load OpenCV library.
- Load the trained face recognition model/dataset.
- Load the face detection model.
- Set recognition threshold.
- Set camera resolution and processing parameters.
- Open live camera using VideoCapture.

2. CHECK CAMERA

- IF camera is not opened THEN
 - Display "Camera Error".
 - STOP.
- END IF

3. CONTINUOUS PROCESSING LOOP

WHILE camera is open:

- Capture one frame from the camera.

- IF frame is not available THEN
 - BREAK.
- END IF

4. PREPROCESSING

- Resize frame if necessary to reduce computation.
- Convert frame to grayscale or the required model format.
- Apply noise reduction if required.
- Improve contrast using histogram equalization/CLAHE when needed.

5. FACE DETECTION

- Detect all faces in the current frame.
- Store the bounding box of every detected face.

6. PROCESS EACH FACE

FOR each detected face:

- Crop the face region from the frame.
- Resize the face to the recognition model input size.
- Normalize the face image.

7. FEATURE EXTRACTION

- Extract facial features/embedding from the cropped face.

8. RECOGNITION

- Compare extracted features with registered features.
- Find the closest matching registered identity.
- Calculate confidence or distance.

```
IF confidence satisfies recognition threshold THEN
    Set identity = matched person's name.
ELSE
    Set identity = "Unknown".
END IF
```

9. VISUALIZATION

```
Draw bounding rectangle around the face.
Draw identity label above/below the rectangle.
Display confidence when required.
```

```
END FOR
```

10. DISPLAY

```
Show the processed frame in a window.
```

11. EFFICIENT LOOP CONTROL

```
Use frame resizing or process every second/third frame if necessary.
Use tracking between detection frames when appropriate.
Avoid unnecessary model loading inside the loop.
```

```
IF user presses the quit key THEN
    BREAK.
END IF
```

```
END WHILE
```

12. TERMINATION

```
Release camera.
Close all OpenCV windows.
Free allocated resources.
```

```
END
```

REAL-TIME OPTIMIZATION:

- Use a smaller processing resolution.
- Load models only once.
- Detect faces at selected intervals.
- Track faces between detection frames.
- Process multiple detected faces in the same frame.
- Use lightweight models for low-cost computers.

2. Vehicle Detection, Tracking and Counting

A traffic monitoring system must analyze a live video stream to:

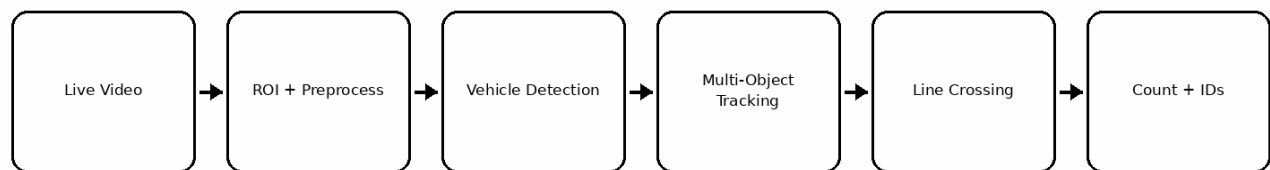
- Detect vehicles entering a predefined region of interest.
- Track multiple vehicles across consecutive frames.
- Count vehicles crossing a virtual line.
- Display bounding boxes, tracking IDs, and vehicle count.

Constraints:

- Avoid double counting.
- Handle multiple vehicles simultaneously.
- Maintain near real-time performance.

Write detailed pseudocode for the complete OpenCV-based vehicle detection, tracking, and counting system. Include video acquisition, preprocessing, detection, tracking, counting logic, visualization, and efficient loop termination.

VEHICLE DETECTION, TRACKING & COUNTING



Illustrative OpenCV pipeline

PSEUDOCODE:

BEGIN

1. INITIALIZATION

- Load OpenCV and required detection/tracking modules.
- Load the vehicle detection model.
- Initialize the multi-object tracker.

Define the Region of Interest (ROI).
Define the virtual counting line.
Set vehicle count = 0.
Create an empty set of counted vehicle IDs.
Open the traffic camera/video using VideoCapture.

2. CHECK VIDEO

IF video is not opened THEN
 Display "Video Error".
 STOP.
END IF

3. CONTINUOUS PROCESSING LOOP

WHILE video is open:

 Read the next video frame.

 IF frame is not available THEN
 BREAK.
 END IF

4. PREPROCESSING

 Resize the frame if required.
 Apply Gaussian/median filtering to reduce camera noise.
 Select/crop the predefined ROI.
 Convert to the required color or model input format.

5. VEHICLE DETECTION

 Run the vehicle detector on the ROI/frame.
 Detect cars, buses, trucks, motorcycles, or other required classes.
 Store bounding boxes and detection confidence values.

6. FILTER DETECTIONS

 Remove detections below the confidence threshold.
 Remove objects outside the required ROI.
 Reject very small or invalid bounding boxes.

7. MULTI-OBJECT TRACKING

 Send valid detections to the tracker.
 Assign a unique tracking ID to each vehicle.
 Update the position of every existing vehicle.
 Maintain previous and current centroid positions.

8. LINE-CROSSING COUNTING

FOR each tracked vehicle:

 Calculate the current centroid.

Retrieve the previous centroid.

Check whether the vehicle moved from one side of the virtual line to the other.

IF the vehicle crosses the virtual line THEN

IF vehicle ID is NOT in counted vehicle IDs THEN
vehicle count = vehicle count + 1.
Add vehicle ID to counted vehicle IDs.
END IF

END IF

END FOR

9. VISUALIZATION

Draw the ROI boundary.
Draw the virtual counting line.
Draw a bounding box around each tracked vehicle.
Display the tracking ID.
Display vehicle count.
Display detection/violation information when required.

10. DISPLAY

Show the processed frame in real time.

11. EFFICIENT LOOP CONTROL

Resize frames for faster processing.
Process detection at selected frame intervals if required.
Track vehicles between detection frames.
Use lightweight detection models when hardware is limited.
Avoid repeated processing of unchanged regions.

IF user presses the quit key THEN
BREAK.
END IF

END WHILE

12. TERMINATION

Release video capture.
Destroy all OpenCV windows.
Release tracker/model resources.

END

DOUBLE-COUNTING PREVENTION:

- Every vehicle receives a unique tracking ID.
- Store counted IDs in a set.
- Increase the count only when an uncounted ID crosses the line.
- Do not count the same ID again even if it remains in the scene.

REAL-TIME OPTIMIZATION:

- Resize input frames.
- Restrict detection to the ROI.
- Detect at selected intervals.
- Track between detections.
- Use confidence and area filtering to reduce unnecessary processing.